COT 3100C: Discrete Structures

Propositional Logic — Lecture Series

Course Overview

This course bridges **programming concepts** with formal mathematics, providing the theoretical foundation that underlies all of computer science. Students will develop rigorous logical reasoning skills applicable to algorithm design, circuit analysis, and software verification.

- Required Prerequisite: **COP 2800 — Java Programming**
- Required Corequisite: **COP 3530 — Data Structures & Algorithms**

Registration Parameters

COT 3100C is a core requirement in the Computer Science curriculum. Enrollment requires demonstrated proficiency in Java and concurrent enrollment in Data Structures.

Students must satisfy both the prerequisite and corequisite conditions to register for this course.

Made with GAMMA

Roadmap: From Language to Logic

This lecture follows a structured five-part progression, moving from the informal ambiguity of everyday English to the precise symbolic machinery of propositional logic.

1

Part 1

The ambiguity of natural English vs. mathematical precision

2

Part 2

Propositions, Boolean variables, and basic connectives

3

Part 3

Core theory of implications, truth tables, and equivalence

4

Part 4

Worked Case Study: Simplifying code conditionals

5

Part 5

Computational limitations and active learning

Made with GAMMA

Lecture Learning Outcomes

By the end of this lecture, students will be able to demonstrate the following competencies in propositional logic:

1

Classify

Identify and classify English statements as formal propositions, distinguishing them from questions, commands, and paradoxes.

2

Construct

Build complete truth tables for compound logical statements involving multiple connectives and propositional variables.

3

Evaluate

Systematically evaluate binary truth values for all possible combinations of input variables in a logical expression.

4

Apply

Use logical equivalences to simplify conditional code branches and digital circuit designs.

Made with GAMMA

The Ambiguity of Natural Language

Natural language is filled with underlying ambiguity. The same sentence can carry multiple valid interpretations depending on context, tone, and convention – a fundamental problem that formal logic was designed to solve.

"You may have cake, or you may have ice cream."

Ambiguity: Does "or" permit both options simultaneously, or is it a strict choice between one or the other? In everyday English, this is often interpreted as exclusive – but mathematically, it need not be.

"If pigs can fly, then your account won't get hacked."

Ambiguity: Is this a genuine security guarantee? In English, implications suggest causality. In logic, the truth of this statement depends only on the truth values of its components – not on any physical connection between them.

"Every American has a dream."

Ambiguity: Do all Americans share a single collective dream, or does each individual possess their own unique dream? The quantifier "every" is deeply ambiguous in natural language but has a precise meaning in formal logic.

Made with GAMMA

The Formal Language of Logic

The Problem with Natural Language

Natural language evolved for communication between humans who share cultural context, tone, and implicit understanding. It was never designed for mathematical precision. A single sentence can mean different things to different readers, making it unsuitable as a foundation for rigorous reasoning.

The Formal Solution

Mathematicians use a **precise, symbolic language** to talk about relationships. This language borrows everyday words like **"or," "implies,"** and **"for all"** – but gives each one an **unambiguous, mathematical definition**.

⚠ These formal definitions do **not** always align with daily conversational usage. Students must consciously override intuitive interpretations.

Connective: AND

True only when **both** components are true simultaneously.

Connective: OR

True when **at least one** component is true (inclusive by default).

Connective: IMPLIES

False **only** when the hypothesis is true and the conclusion is false.

Quantifier: FOR ALL

The statement must hold for **every** element in the domain.

Made with GAMMA

How You Already Use Logic

Modern programming languages implement propositional logic directly. Every conditional branch in Java evaluates a logical expression down to a single **Boolean value** – the same binary truth values studied in formal logic.

Java Logical Operators

&& — Logical AND

Evaluates to `true` only when **both** operands are `true`. Short-circuits: if the left operand is `false`, the right is never evaluated.



|| — Logical OR

Evaluates to `true` when **at least one** operand is `true`. Short-circuits: if the left operand is `true`, the right is never evaluated.



! — Logical NOT

Inverts the truth value of its operand. `true` becomes `false` and vice versa. Applied to a single Boolean expression.

Key Insight

These operators evaluate complex expressions down to a single **Boolean value**. No matter how many variables or nested conditions appear in a Java expression, the result is always exactly one of two values: `true` or `false`.

This binary reduction is the computational manifestation of propositional logic – the same system we will formalize mathematically throughout this course.

Made with GAMMA

Computational Truth Values

Consider the following Java program segment. Trace the evaluation step by step to see how formal logic operates inside running code.

Java Code Segment

```
boolean isHungry = true;
boolean isGrumpy = false;
boolean state = isHungry && !isGrumpy;
```

Step-by-Step Evaluation

01

Evaluate `!isGrumpy`

`isGrumpy` is `false`, so `!isGrumpy` evaluates to `true`.

02

Evaluate `isHungry && true`

`isHungry` is `true`, and `true && true` evaluates to `true`.

03

Assign result

The variable `state` is assigned the value `true`.

Core Principle: Binary Truth

The variable `state` evaluates strictly to **true**. There is no intermediate value, no "mostly true," and no ambiguity.

Logical statements are strictly **binary** – they are never "partially true," "probably true," or "true in some contexts." This is the foundational constraint of propositional logic and the reason it can be implemented directly in digital hardware.

> Every Boolean expression in every programming language ultimately reduces to a single bit: 0 or 1, false or true.

Made with GAMMA

The Dual Meanings of "OR"

One of the most important distinctions in propositional logic is the difference between two interpretations of the word "or." Natural language uses both interchangeably; formal logic requires a precise choice.

Inclusive OR (Mathematical Default)

True if **at least one** component is true – including the case where **both** are true.

> "You can have cake, or you can have ice cream (or both)."

In formal logic and in Java's `||` operator, OR is always inclusive. The expression `P || Q` is true whenever P is true, Q is true, or both are true. It is false only when both P and Q are false.

Exclusive OR (XOR)

True if **exactly one** component is true, but **not both**. This matches the everyday English sense of "either/or."

> "You can have cake, or you can have ice cream – but not both."

XOR is denoted ⊕ in mathematics and `^` in Java for booleans. It is true only when the two operands differ in truth value.

P	Q	P OR Q (Inclusive)	P XOR Q (Exclusive)
T	T	T	F
T	F	T	T
F	T	T	T
F	F	F	F

Mathematicians default to **inclusive OR**. Unless explicitly stated otherwise, "or" in a formal context always means inclusive.

Made with GAMMA

Causality vs. Truth Value

The mathematical implication is one of the most counterintuitive concepts in propositional logic because it diverges sharply from the English meaning of "if…then."

English Implication

In everyday language, "if P then Q" assumes a **physical cause-and-effect connection** between P and Q. The hypothesis must be the reason the conclusion holds.

> "If you follow the security protocol, your account won't get hacked."

Here, following the protocol is understood to *cause* the account's safety. The two events are meaningfully related.

Mathematical Implication

In formal logic, "if P then Q" cares strictly about the **truth values** of P and Q – not about any causal relationship between them.

> "If pigs can fly, then your account won't get hacked."

This mathematical implication is completely **True** – because pigs cannot fly, the hypothesis P is false, and a false hypothesis makes the entire implication vacuously true.

P (Hypothesis)	Q (Conclusion)	P → Q (Implication)	Interpretation
T	T	T	Hypothesis true, conclusion true – implication holds
T	F	F	Hypothesis true, conclusion false – implication **fails**
F	T	T	Hypothesis false – implication vacuously true
F	F	T	Hypothesis false – implication vacuously true

Made with GAMMA

The Core Building Block: Propositions

Formal Definition

A **proposition** is a declarative statement that is either strictly **True (T)** or strictly **False (F)**, but cannot be both simultaneously and cannot be neither.

This binary constraint is absolute. Propositions exclude:

- Questions: *"Is it raining?"*
- Commands: *"Close the door."*
- Paradoxes: *"This statement is false."*
- Opinions without objective truth: *"Pizza is delicious."*

Examples of Valid Propositions

"All humans are mortal." — True

A universally accepted biological fact. It has a definite, objective truth value.

"2 + 3 = 5." — True

A mathematical identity. Its truth value is determined by arithmetic, not interpretation.

"2 + 3 = 6." — False

Incorrect, but still a **valid proposition**. A proposition need not be true – it only needs to have a definite truth value. False statements are propositions too.

Made with GAMMA

Symbolic Abstraction: Propositional Variables

To reason about logical structure independently of specific content, we replace concrete sentences with abstract symbols. This abstraction is the gateway from informal reasoning to formal logic.

From Sentences to Symbols

We substitute specific sentences with variables like **P** and **Q**. These are called **propositional variables** or **Boolean variables**. They are restricted to taking on only the values **T** and **F**.

Let P = "It is raining."

P can be T or F depending on the weather. The variable abstracts away the content.

Let Q = "The ground is wet."

Q is independently T or F. We can now write P → Q without referring to weather at all.

Compound: P ∧ Q, P ∨ Q, ¬P

Once abstracted, we apply connectives symbolically and evaluate truth tables for any combination.

George Boole (1815–1864)

Propositional variables are named after the 19th-century mathematician **George Boole**, who first formalized the algebra of logic in his 1854 work *An Investigation of the Laws of Thought*.

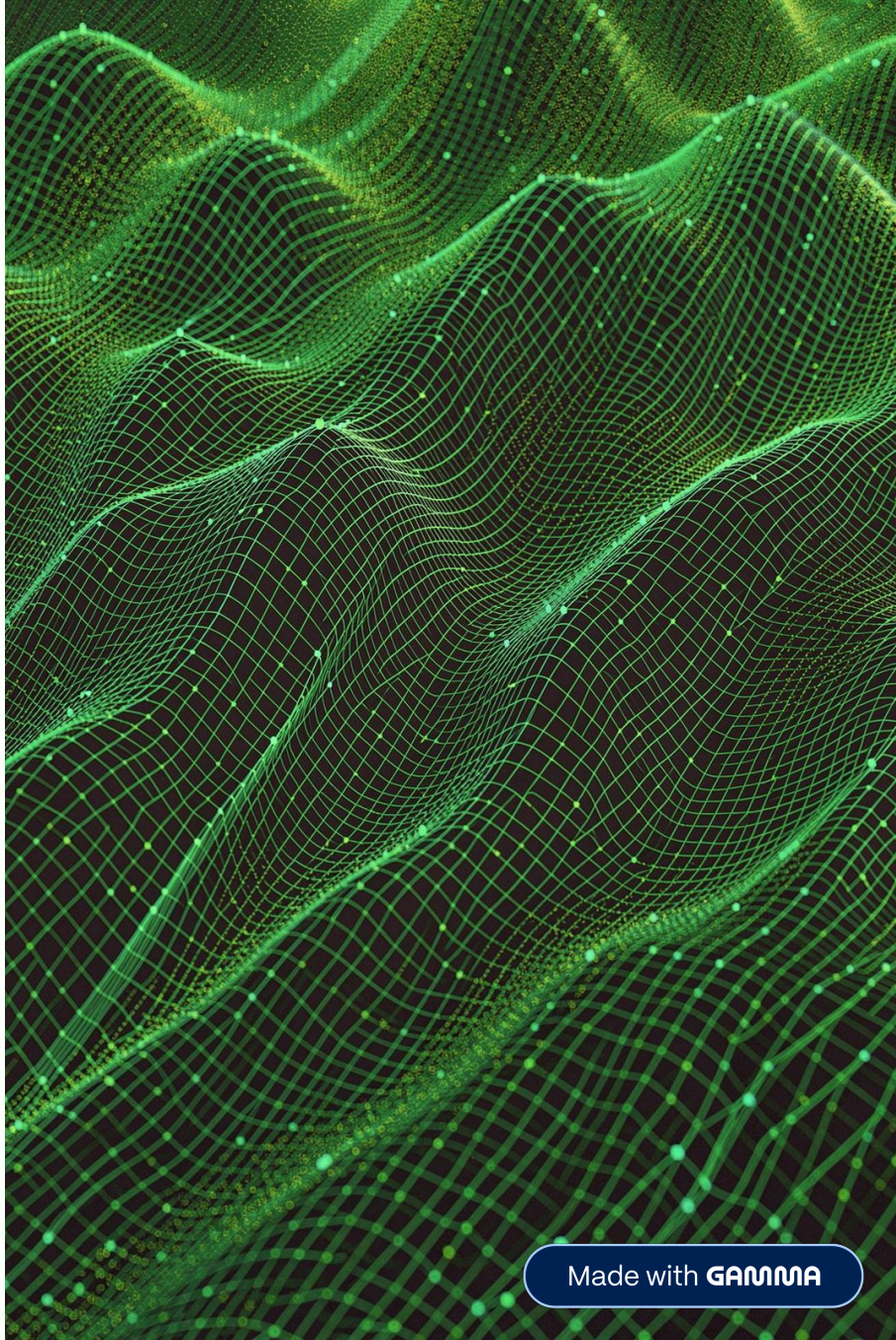
Boole demonstrated that logical reasoning could be expressed algebraically, treating truth values as numbers (1 and 0) and logical operations as arithmetic. His system became the mathematical foundation for all modern digital computing.

> Every transistor in every computer executes Boolean logic – Boole's 19th-century abstraction made physical.

Made with GAMMA

Propositional Logic: Truth Values & Logical Operators

An introduction to the mathematical foundation of digital computing – discrete, precise, and binary.



Made with GAMMA

The Principle of Binary Constraint

The Domain

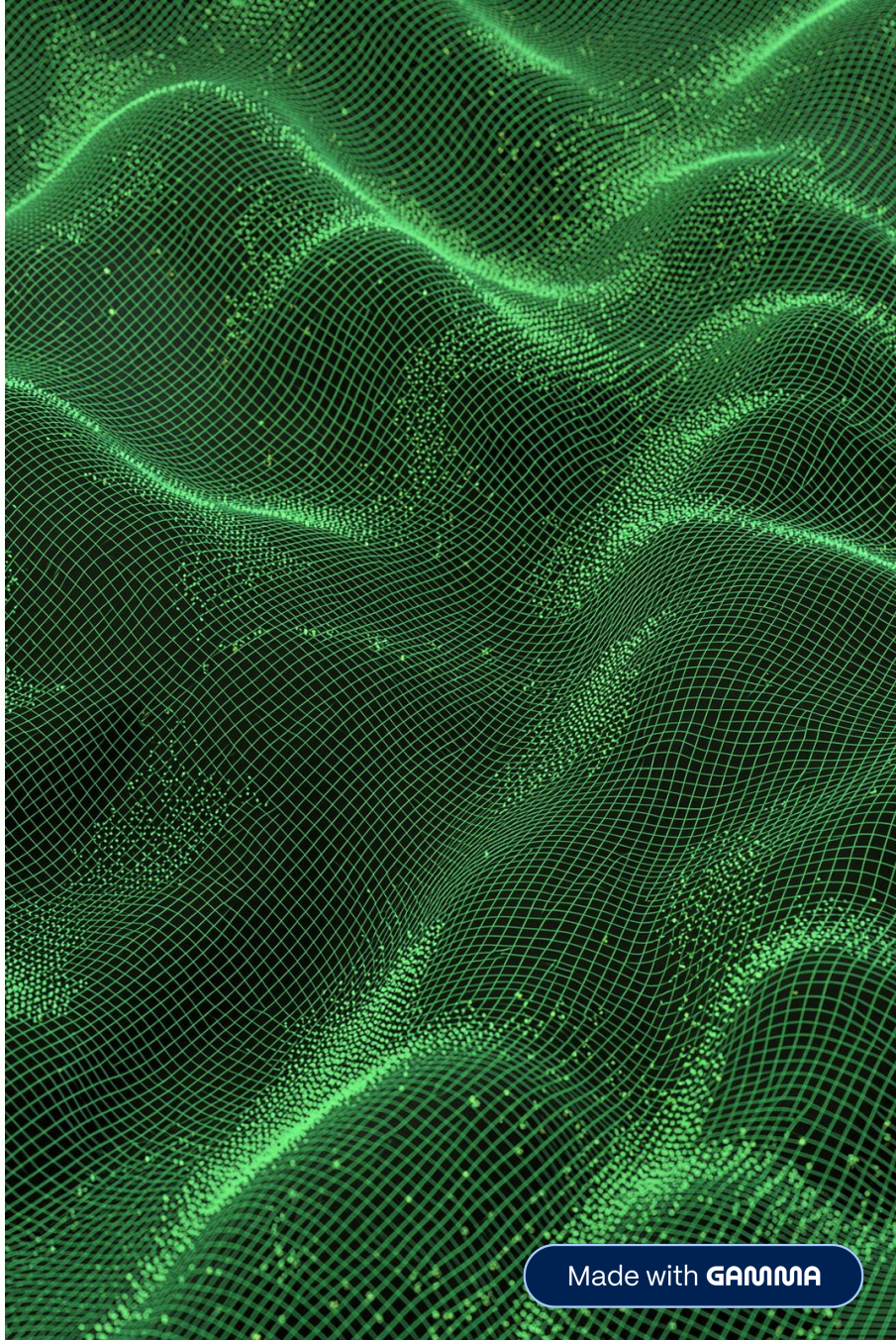
Propositional variables belong strictly to □ **= {T, F}** – no middle ground, no ambiguity.

Exactly One Value

Every variable must take on **exactly one** of these values at any given moment.

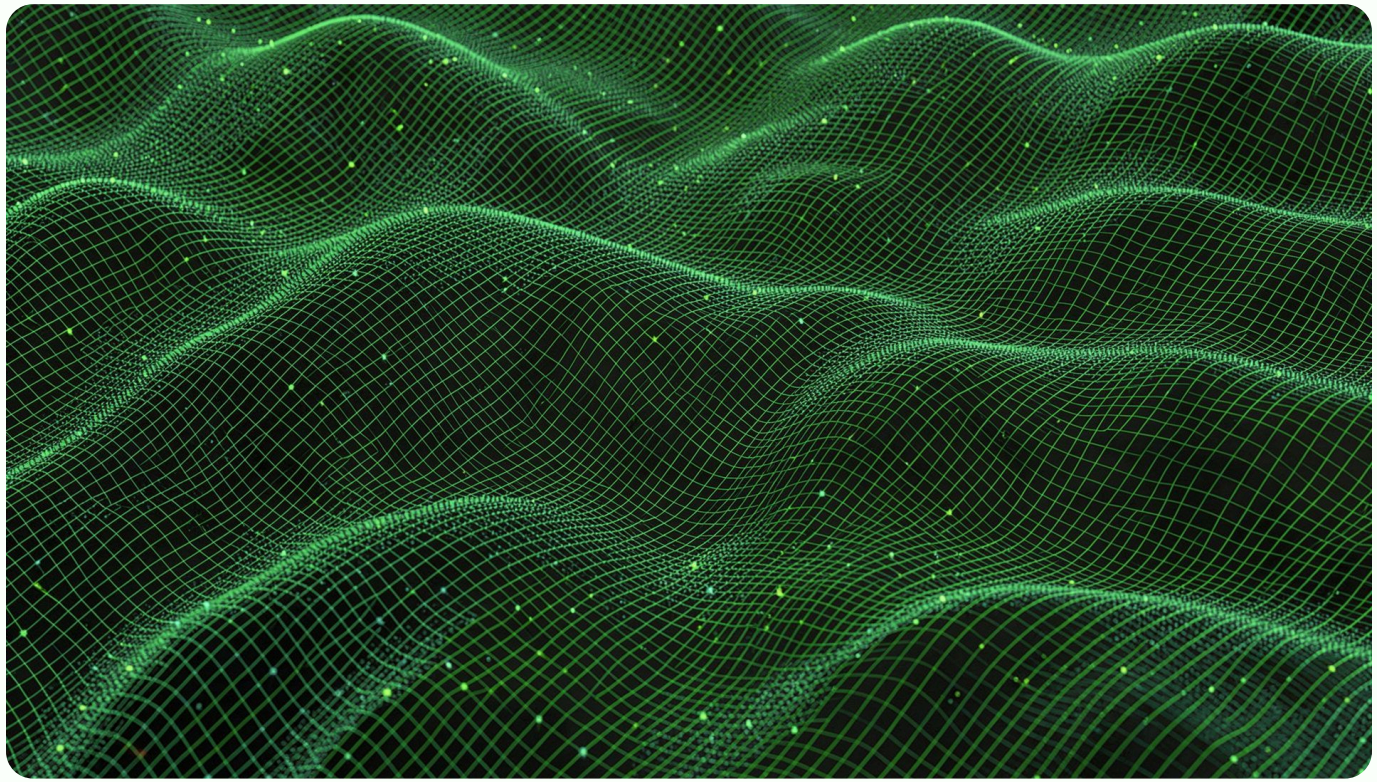
Foundation of Computing

This binary constraint forms the mathematical basis for all classical **digital computers**.



Made with GAMMA

Mathematical Logical Operators



Building Compound Propositions

We combine simple propositions into larger structures using **logical connectives**.

Two input variables – P and Q – enter a connective operation and produce a single compound output proposition.

Core Operators:

- **NOT** – Negation
- **AND** – Conjunction
- **OR** – Disjunction

Made with GAMMA

The Negation Operator

What It Does

Operation: Reverses the truth value of a proposition.

Notation: ¬P (also written P̄ or NOT(P))

Negation is the simplest unary operator – it takes a single input and flips it.

In Java: `!P` performs logical negation.

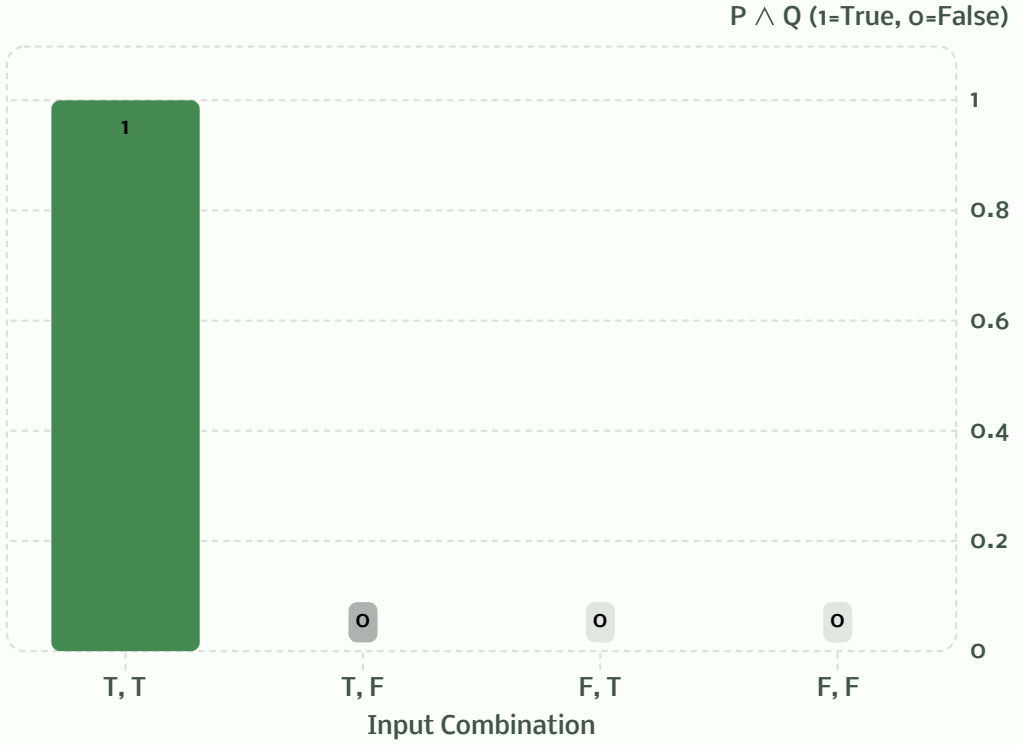
P	¬P
T	**F**
F	**T**

The output is always the **inverse** of the input.

Made with GAMMA

Logical Conjunction (AND)

Operation: True only when **both** components are true. Notation: P ∧ Q. In Java: `P && Q`



P	Q	P ∧ Q
T	T	**T**
T	F	F
F	T	F
F	F	F

Conjunction is true in **exactly one case**: when both inputs are true.

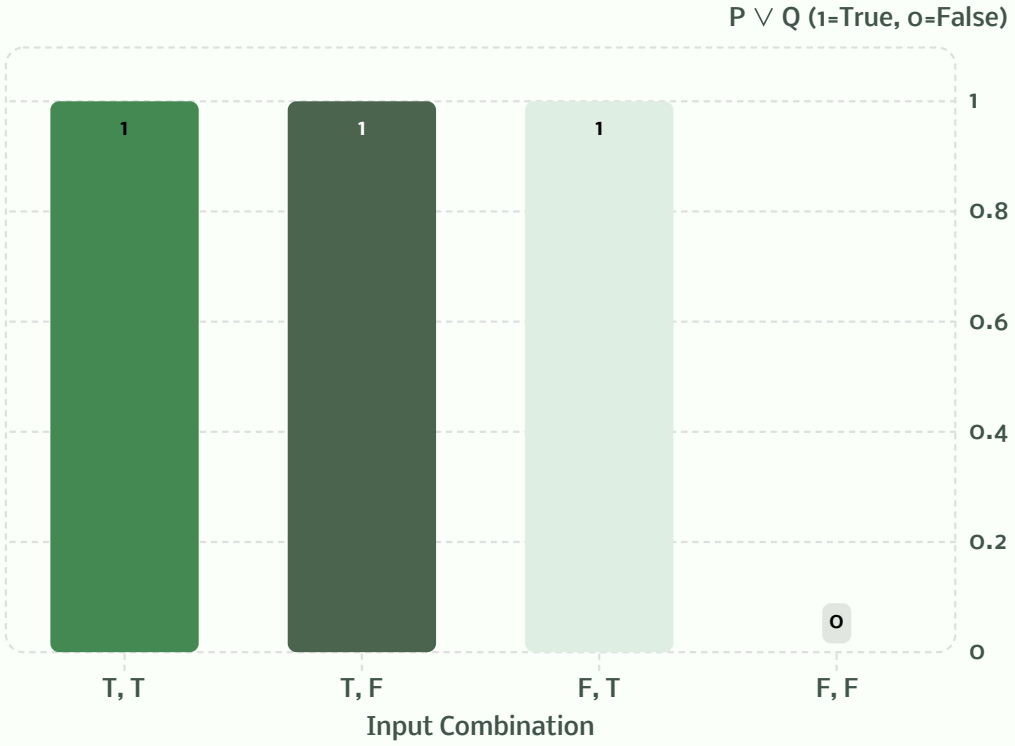
Made with GAMMA

Inclusive Disjunction (OR)

Operation: True if **at least one** component is true. Notation: P ∨ Q. In Java: `P || Q`

P	Q	P ∨ Q
T	T	**T**
T	F	**T**
F	T	**T**
F	F	F

OR is false in **exactly one case**: when both inputs are false.



Made with GAMMA

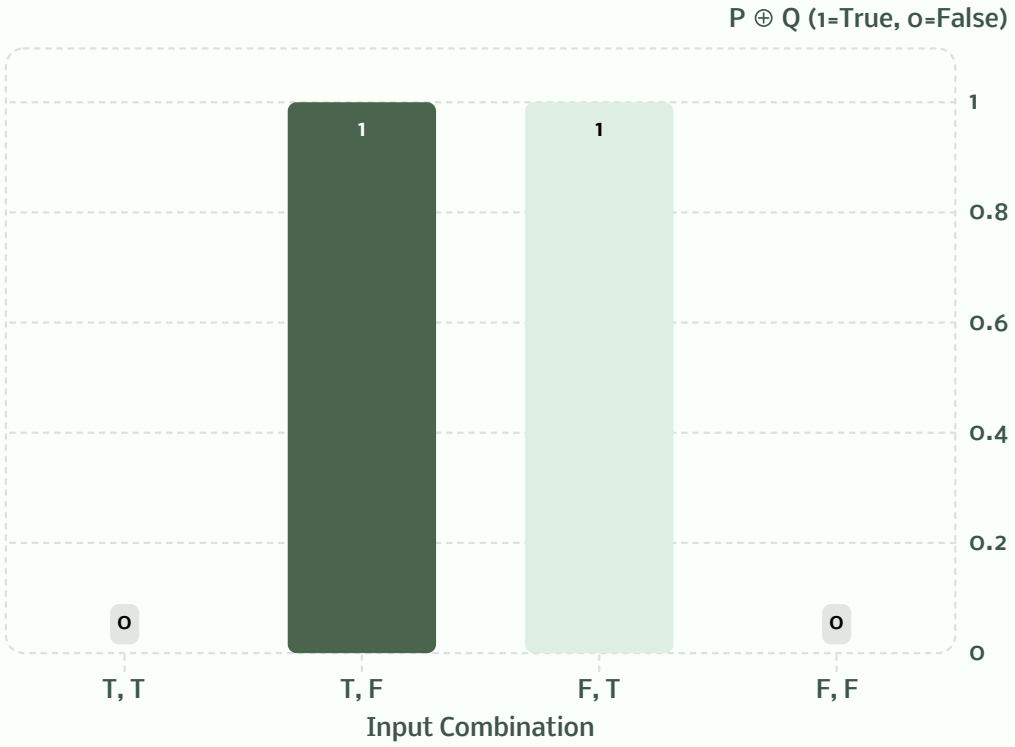
Exclusive Disjunction (XOR)

The "Difference" Operator

Operation: True if **exactly one** component is true. False when both match.

Notation: P ⊕ Q – in Java: `P ^ Q`

P	Q	P ⊕ Q
T	T	F
T	F	**T**
F	T	**T**
F	F	F



XOR is true when inputs **differ**, false when inputs are **identical**.

Made with GAMMA

Logical Translation Key

Every logical operator has an equivalent form across English, mathematics, and Java. Use this reference table to translate between them fluently.

English	Mathematical Notation	Java Syntax
NOT P	¬P	`!P`
P AND Q	P ∧ Q	`P && Q`
P OR Q	P ∨ Q	`P || Q`
P XOR Q	P ⊕ Q	`P ^ Q`

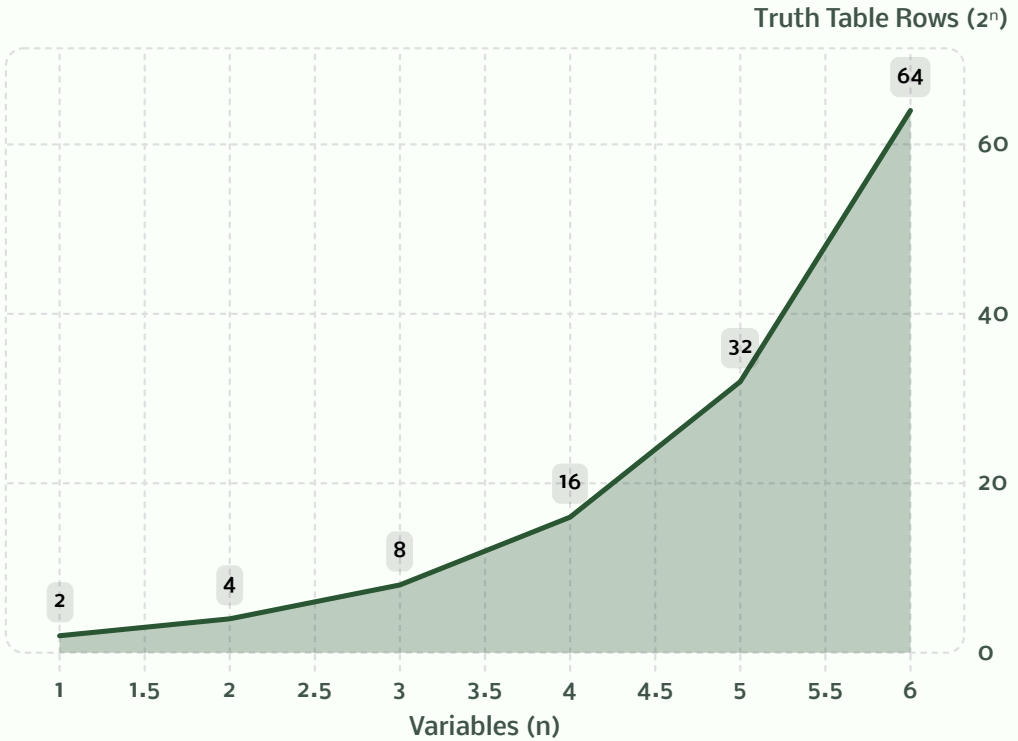
Made with GAMMA

The Exponential Growth of Combinations

Why Truth Tables Scale Fast

A truth table must list **every possible combination** of truth values. For **n** independent variables, the total number of rows is:

$$2^n \text{ rows}$$



2 vars

4 rows

3 vars

8 rows

4 vars

16 rows

Made with GAMMA

The Alternating Pattern Algorithm

Fill Truth Tables Without Errors

Use a **binary pattern** to enumerate all 2^n combinations systematically.

P	Q	R
T	T	T
T	T	F
T	F	T
T	F	F
F	T	T
F	T	F
F	F	T
F	F	F

Pattern blocks: **4/4 → 2/2 → 1/1**

First Column (P)

Alternate blocks of T/F every **2^(n−1)** rows. For 3 variables: 4 T's, then 4 F's.

Middle Column (Q)

Alternate every **2^(n−2)** rows. For 3 variables: 2 T's, then 2 F's, repeat.

Last Column (R)

Alternate **every single row**: T, F, T, F, T, F, T, F.

Made with GAMMA

Logical Implication

If P, Then Q

Operation: Asserts that if the premise occurs, the conclusion must follow.

Notation: P → Q – read as "P implies Q"

- **Hypothesis (P):** The "if" condition – the premise
- **Conclusion (Q):** The "then" result – what must follow

A false hypothesis makes the implication vacuously **true** – this is the most counterintuitive rule in propositional logic.

P	Q	P → Q
T	T	**T**
T	F	F
F	T	**T**
F	F	**T**

Implication is only **false** when the hypothesis is true but the conclusion is false.

Made with GAMMA

Logic: Implication and Biconditional

A structured introduction to conditional operators, truth tables, logical equivalence, and practical applications in computer science and formal reasoning.

1

Implication

P → Q and the false-hypothesis convention

2

Biconditional

P ↔ Q and decomposition into two directions

3

Equivalence

Substitution, proofs, and logical identity

4

Applications

System specifications and code optimization

Made with GAMMA

IMPLICATION

The Implication Truth Table

The logical operator **P → Q** (read: "if P then Q") is one of the most fundamental connectives in formal logic. Its truth table contains a single defining rule: the implication is **false only when a true hypothesis leads to a false conclusion**.

P	Q	P → Q	Row
T	T	**T**	(tt)
T	F	**F**	(tf) ← only false row
F	T	**T**	(ft)
F	F	**T**	(ff)

The Contract Analogy

Think of P → Q as a contractual promise. The contract is **only broken** when you meet the stated condition (P is True) but fail to deliver the promised outcome (Q is False). In all other scenarios – including when the condition was never triggered – the contract holds.

- Row (tt): Condition met, promise kept → **True**
- Row (tf): Condition met, promise broken → **False**
- Row (ft): Condition not met → **True**
- Row (ff): Condition not met → **True**

Made with GAMMA

IMPLICATION

The Convention of False Hypotheses

A frequently misunderstood feature of formal logic is what happens when the hypothesis P is false. The mathematical convention is clear: **when P is false, P → Q is always true**, regardless of the value of Q.

Formal Reasoning

When P is false, we are necessarily on rows **(ft)** or **(ff)** of the truth table – both of which evaluate to True. The rationale is straightforward: the starting condition was *never met*, so the promise encoded in the implication has never been broken. You cannot violate a contract whose triggering condition never occurred.

Row (ft): P = F, Q = T

P → Q evaluates to **True**

Row (ff): P = F, Q = F

P → Q evaluates to **True**

Concrete Example

> "If you run 100 miles, I will pay you $1,000."

Suppose you did *not* run 100 miles. The condition (P) is false. Has the promise been broken? **No** – there is nothing to invalidate. The statement remains vacuously true. The obligation to pay $1,000 was conditional on something that never happened.

This is called a **vacuously true** implication – true not because the conclusion holds, but because the premise was never triggered.

Made with GAMMA

IMPLICATION

Non-Causal Implications

A critical insight for students of formal logic: **mathematical implication does not require causality**. The operator P → Q is evaluated purely on truth values – not on whether P has any logical, physical, or semantic connection to Q.

Causal Example

> "If you practice coding, you will improve."

Here, a genuine cause-and-effect relationship exists between the hypothesis and conclusion. This is the kind of implication we encounter in everyday reasoning. However, formal logic does *not* require this relationship.

Non-Causal Example

> "If $1 + 1 = 2$, then Tokyo is in Japan."

There is no causal link between arithmetic and geography. Yet both propositions are individually True, placing this on row **(tt)** of the truth table – making the implication **True**. Formal logic accepts this without question.

Why This Matters

Stripping away causality makes logic a **universal tool**. Computer systems do not reason about meaning or intent – they evaluate conditions and values. This simplicity is precisely what makes propositional logic so powerful for engineering and computation.

The Formal Rule

Whenever both P and Q are independently True, the implication P → Q is True – **regardless of any relationship** between them. Logic cares only about rows in the truth table, not about real-world connections.

Engineering Consequence

This property allows engineers to build **rule-based systems** that evaluate conditions mechanically. No semantic understanding is needed – only truth value assignments and table lookups.

Made with GAMMA

IMPLICATION

Evaluating Implications with Unknown Premises

A powerful feature of formal logic is the ability to determine the truth value of an implication **even when the hypothesis is unknown** – provided the conclusion has a known value.

> "If Goldbach's Conjecture is true, then $x^2 \geq 0$ for every real number x."

Hypothesis (P)

Goldbach's Conjecture is true.

Status: **Unknown** – as of today, this conjecture has neither been proven nor disproven. Mathematicians have verified it for enormous numbers, but no general proof exists.

Conclusion (Q)

$x^2 \geq 0$ **for every real number** x**.**

Status: **True** – this is a well-established mathematical fact. Squaring any real number always yields a non-negative result.

Overall Evaluation

Because Q is **True**, we are on either row **(tt)** or row **(ft)**. In both cases, P → Q evaluates to **True**.

The unknown status of P is irrelevant – the truth of the conclusion guarantees the truth of the implication.

Key takeaway: When the conclusion Q is True, the implication P → Q is always True – regardless of whether P is known, unknown, true, or false.

Made with GAMMA

BICONDITIONAL

The "If and Only If" Operator

The biconditional operator **P ↔ Q** (also written **P IFF Q**) asserts something stronger than a simple implication: it claims that the two propositions share the *identical* truth value under all circumstances. Either both are True, or both are False – with no exceptions.

The Biconditional Truth Table

P	Q	P ↔ Q
T	T	**T**
T	F	**F**
F	T	**F**
F	F	**T**

True only on rows 1 and 4 – when P and Q have matching truth values.

Mathematical Example

> For every real number x: $x^2 - 4 = 0$ IFF $|x| = 2$

Both sides of this biconditional are true for exactly the same values of x (namely $x = 2$ and $x = -2$), and both are false for all other values. Neither can be true while the other is false.

Relation to XOR

The biconditional is the **exact logical inverse** of the exclusive OR (XOR) operator. Where XOR outputs True when values differ, IFF outputs True when values match.

$$P \leftrightarrow Q \equiv \neg(P \oplus Q)$$

Made with GAMMA

BICONDITIONAL

Decomposing the Biconditional

The biconditional is not a primitive – it can be fully decomposed into two implications joined by conjunction. This decomposition is both practically useful and theoretically important for constructing proofs.

The Fundamental Equivalence

$$P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$$

In words: "P if and only if Q" is logically equivalent to "P implies Q, **AND** Q implies P." Both directions of the implication must hold simultaneously.

Proof Verification by Truth Table

P	Q	P → Q	Q → P	(P→Q) ∧(Q→ P)	P ↔ Q
T	T	T	T	**T**	**T**
T	F	F	T	**F**	**F**
F	T	T	F	**F**	**F**
F	F	T	T	**T**	**T**

Proof Strategy

To establish an IFF statement in mathematics, you must prove **both directions** independently: first show P → Q, then show Q → P. Each is its own sub-proof.

Logical Identity

The truth columns of P ↔ Q and (P → Q) ∧ (Q → P) match **exactly on every row**. They are logically identical – one may always be substituted for the other.

Made with GAMMA

EQUIVALENCE

Logical Equivalence

Logical equivalence is one of the most powerful tools in both theoretical logic and applied computer science. Two expressions are **logically equivalent** if and only if their truth values agree on every single row of a joint truth table – with no exceptions.

Formal Definition

Two logical expressions **E_1** and **E_2** are logically equivalent, written **$E_1 \equiv E_2$**, if for every possible assignment of truth values to their component variables, E_1 and E_2 produce the same output.

$$E_1 \equiv E_2 \iff (E_1 \leftrightarrow E_2) \text{ is a tautology}$$

Equivalently, $E_1 \equiv E_2$ when the biconditional $E_1 \leftrightarrow E_2$ is true on every row – that is, when it is a *tautology*.

Why It Matters

1 Substitution in Proofs

Equivalent expressions can replace one another anywhere in a formal proof without changing its validity.

2 Code Simplification

Equivalent boolean expressions in code can be swapped to improve readability or runtime performance.

3 Circuit Design

Equivalent gate configurations allow engineers to reduce component count while preserving behavior.

Logical equivalence (≡) is a **meta-level** statement about two expressions – it lives outside the truth table itself. The biconditional (↔) is an **object-level** operator that appears *inside* the table. The distinction matters in formal proofs.

Made with GAMMA

CASE STUDY

Case Study: Verifying System Specifications

This case study demonstrates why the convention of false hypotheses is not merely a mathematical curiosity – it has direct, critical implications for the correctness of real engineering systems.

System Setup

A control system is governed by **12 rules**, each of the form: "If sensor condition C_i occurs, take action A_i."

The complete specification requires **all 12 rules to be satisfied simultaneously**:

$$(C_1 \to A_1) \wedge (C_2 \to A_2) \wedge \cdots \wedge (C_{12} \to A_{12})$$

For the system to operate correctly, this entire conjunction must evaluate to **True**.

A Typical Scenario

Suppose at a given moment, **only C_2 and C_5 are active** (their sensor conditions are firing). All other conditions C_i for $i \neq 2, 5$ are False.

What Happens to the Inactive Rules?

For all inactive sensors, we have $C_i = F$. Each corresponding rule $C_i \to A_i$ has a **false hypothesis**. By the standard convention, each such implication evaluates to **True** – regardless of whether the action A_i is also true or false.

Why This Is Critical

If false hypotheses evaluated to **False**, then every inactive rule would contribute a False to the conjunction – making the **entire specification fail** whenever any sensor is inactive. The system would never satisfy its spec in normal operation. The vacuous-truth convention is what makes rule-based systems **logically coherent**.

Made with GAMMA

CASE STUDY

Case Study: Code Optimization

Logical equivalence provides a rigorous, mechanical method for simplifying boolean expressions in software – reducing computational overhead while provably preserving program behavior.

Original Code

```
if ( x > 0 || (x <= 0 && y > 100) ) {
   // execute block
}
```

Step 1: Assign Propositional Variables

Let **A** = "x > 0" and **B** = "y > 100".

Observe that `x <= 0` is precisely the negation of `x > 0`, so it is written as **¬A**.

Step 2: Translate to Logic

$$A \vee (\neg A \wedge B)$$

This is the formal propositional representation of the original boolean condition.

Step 3: Simplify via Equivalence

Apply the **Absorption Law** (a standard logical equivalence):

$$A \vee (\neg A \wedge B) \equiv A \vee B$$

A	B	A∨(¬A∧B)	A∨B
T	T	**T**	**T**
T	F	**T**	**T**
F	T	**T**	**T**
F	F	**F**	**F**

Optimized Code

```
if ( x > 0 || y > 100 ) {
   // execute block
}
```

The redundant sub-expression `(x <= 0 && y > 100)` is eliminated. Behavior is **provably identical** on all inputs.

Made with GAMMA

Key Takeaways

This module covered the two conditional operators at the heart of propositional logic, together with the concept of logical equivalence that makes formal reasoning a practical engineering tool.

1

Implication P → Q

False *only* on row (tf). The contract analogy: the only violation is a met condition with an undelivered promise. All other rows, including both false-hypothesis rows, evaluate to True.

2

Vacuous Truth

When P is false, P → Q is always True. This is not a quirk – it is a necessary convention that makes rule-based systems, specifications, and conditional reasoning coherent in practice.

3

Non-Causality

Implication is evaluated purely on truth values. No causal or semantic relationship between P and Q is required – a property that makes formal logic universally applicable to computational systems.

4

Biconditional P ↔ Q

True when P and Q share the same truth value; equivalent to (P → Q) ∧ (Q → P). To prove an IFF statement, both directions must be established independently.

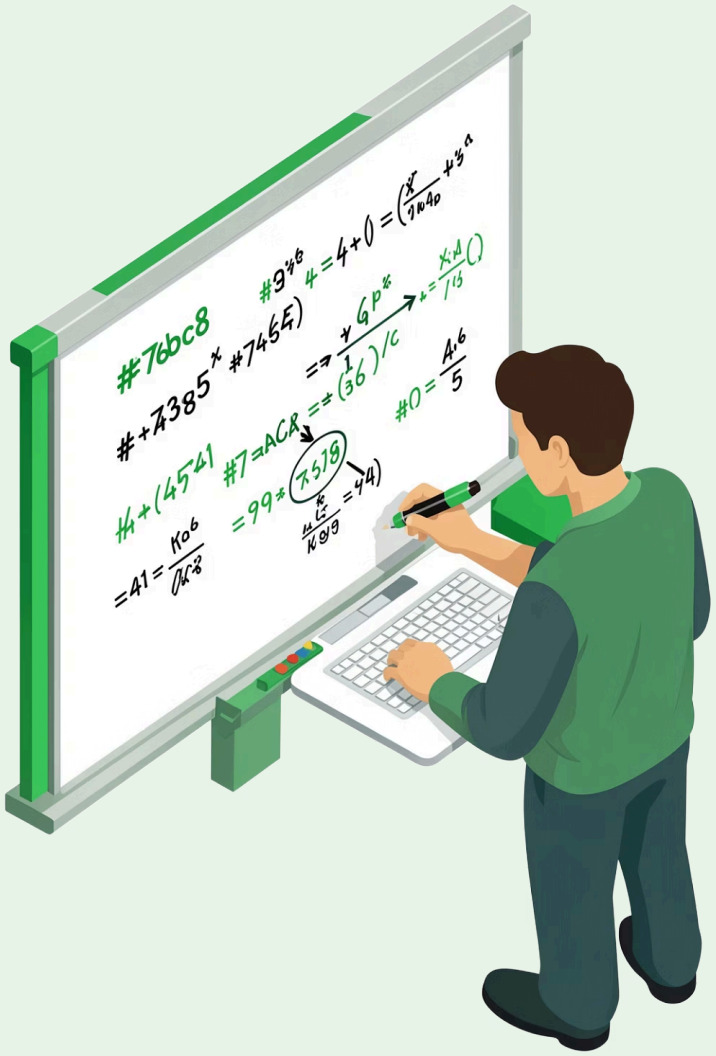
5

Logical Equivalence ≡

Two expressions are equivalent when they agree on every row of a truth table. Equivalent expressions are freely interchangeable in proofs and code – enabling simplification without loss of correctness.

Mastery of implication and equivalence underpins virtually every area of computer science: from compiler optimization and digital circuit design to database query rewriting and formal verification of safety-critical systems.

Made with GAMMA



Formulating the Logical Expression

The goal: prove that a complex conditional can be safely reduced to a simpler form.

Original Condition

$A \vee (\neg A \wedge B)$

Proposed Optimization

$A \vee B$

Our Goal

Prove logical equivalence using a truth table

Made with GAMMA

Truth Table Step 1: Evaluating Negation

Set up the base variables **A** and **B**, then compute $\neg A$ by flipping every value in column A – True becomes False, and False becomes True. This is the foundation for all subsequent calculations.

All four possible combinations of A and B must be represented to make the proof complete.

A	B	¬A
T	T	F
T	F	F
F	T	T
F	F	T

Made with GAMMA

Truth Table Step 2: Evaluating Conjunction

A	B	¬A	¬A ∧ B
T	T	F	F
T	F	F	F
F	T	T	**T**
F	F	T	F

The AND Rule

Evaluate the inner clause $\neg A \wedge B$. AND ($\wedge$) is **true only when both inputs are True**. Scanning each row, only row 3 satisfies this – where $\neg A = T$ and $B = T$.

Result column

F, F, **T**, F

Made with GAMMA

Truth Table Step 3: Evaluating Disjunction

Now evaluate the outer OR: $A \vee (\neg A \wedge B)$. OR ($\vee$) is true if **at least one** input is True.

A	B	¬A	¬A ∧ B	A ∨ (¬A ∧ B)
T	T	F	F	**T**
T	F	F	F	**T**
F	T	T	T	**T**
F	F	T	F	F

Result column: **T, T, T, F** – rows 1-3 have at least one True operand.

Made with GAMMA

Truth Table Step 4: Proving Equivalence

Add the final column for $A \vee B$ and compare. When two expressions yield **identical truth values across every possible input**, they are logically equivalent.

Both columns read T, T, T, F – the expressions are proven equivalent!

A	B	A ∨ (¬A ∧ B)	A ∨ B
T	T	T	T
T	F	T	T
F	T	T	T
F	F	F	F

Made with GAMMA

Reasoning by Cases: Case 1 (*A* is True)



When A = True

Any disjunction with a True operand is immediately True – no need to evaluate the rest. Apply this to both expressions:

Both evaluate to **True** ✓

Original

$T \vee (F \wedge B) \equiv T$

Simplified

$T \vee B \equiv T$

Made with GAMMA

Reasoning by Cases: Case 2 (A is False)

When A = False, a disjunction reduces entirely to its other operand: $F \vee x \equiv x$. Trace both expressions:

Simplified Form

$F \vee B \equiv B$



Original Form

$F \vee (T \wedge B) \equiv B$

Both expressions evaluate to **B** in this case ✓ – equivalence is confirmed across all cases.

Made with GAMMA

Deploying the Optimized Code

The abstract proof maps directly to real software. This refactoring is mathematically safe because we just proved it:

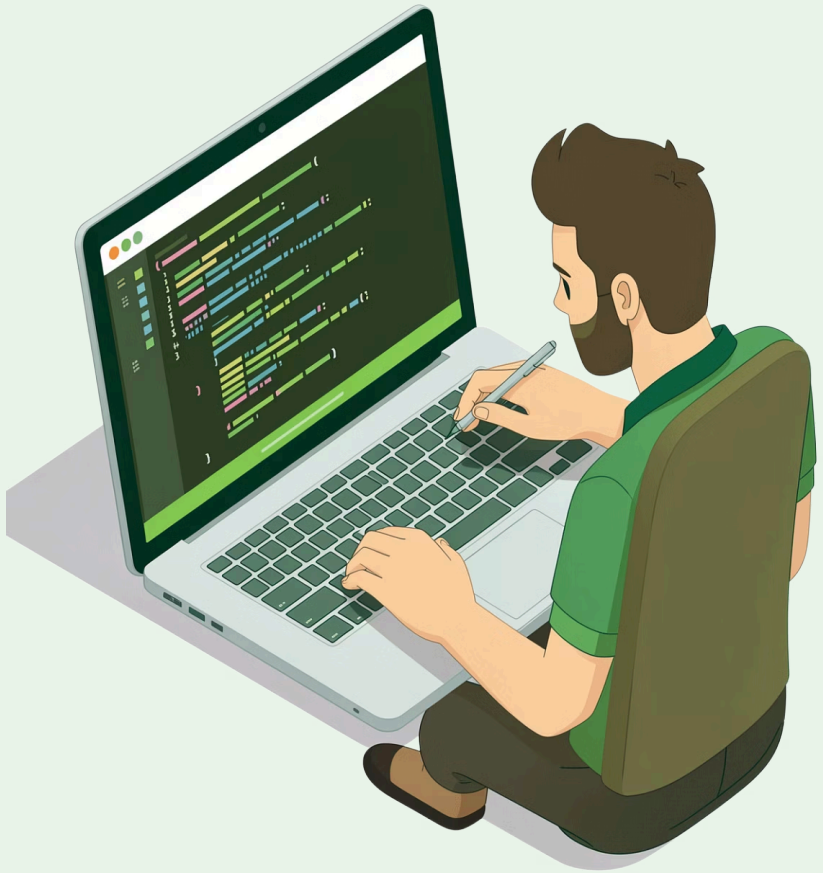
Original Condition

```
if ( x > 0 || (x <= 0 && y > 100) )
```

Optimized Condition

```
if ( x > 0 || y > 100 )
```

This substitution is valid by **propositional equivalence** – not guesswork. Logic is the engineer's license to simplify with confidence.



Made with GAMMA

Why Simplify Logic in Software?



Readability

Simpler boolean expressions are faster to read, easier to debug, and less likely to harbor hidden bugs during maintenance.



Performance

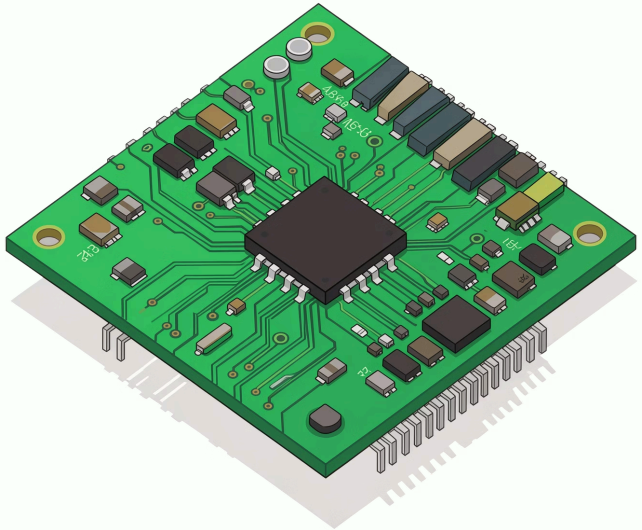
Fewer logical operations mean fewer CPU instructions at runtime – every condition evaluated is work the processor must do.



Compiler Optimization

Modern optimizing compilers apply propositional equivalence rules automatically during the build process – this is the math running under the hood.

Made with GAMMA



REAL-WORLD IMPACT

Logic Minimization in Hardware

Digital circuits are described **directly** by logical formulas – every AND, OR, and NOT maps to a physical gate on silicon.

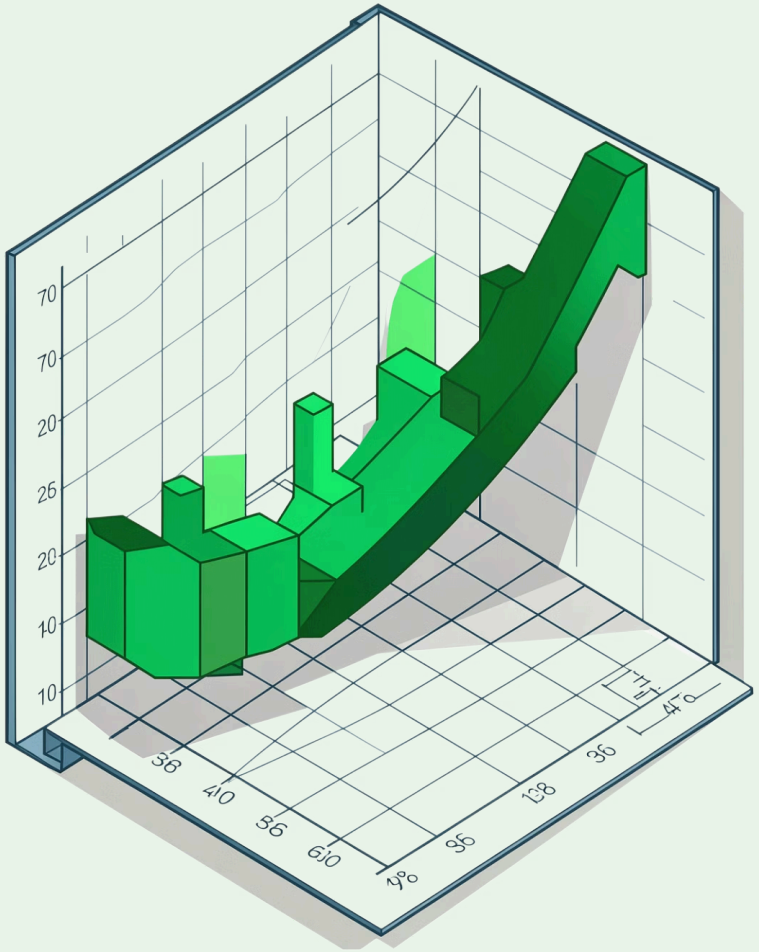
Fewer Gates

Simplified formulas reduce the number of physical logic gates required on a microchip.

Lower Cost & Power

Smaller die area means reduced manufacturing cost and lower power consumption – critical at scale.

Made with GAMMA



The Exponential Scaling Barrier

Truth Table Growth

Tables grow exponentially: **2n rows** for *n* variables

SAT Problem

Deciding if a logical formula can ever evaluate to **True**

30 Variables

A table has over **1 billion rows**!

Open Question

Is there an efficient **polynomial-time** solution?

Made with GAMMA

Complexity and P vs. NP

Revolutionary Upside

An efficient SAT solution would revolutionize **scheduling, routing, and verification**

Catastrophic Downside

It would **break global cryptography**, making transactions insecure

P vs. NP

Is SAT solvable in polynomial time? The defining open problem of computer science.

$1M

The Prize

Solve P vs. NP and claim the **$1,000,000** Millennium Prize!

Made with GAMMA

Logical Fallacies: Converse

Implication

$P \rightarrow Q$

"If I am hungry, then I am grumpy."

Converse

$Q \rightarrow P$

"If I am grumpy, then I am hungry."

Crucial Rule: The implication and its converse are **not equivalent**! The converse is not necessarily true.



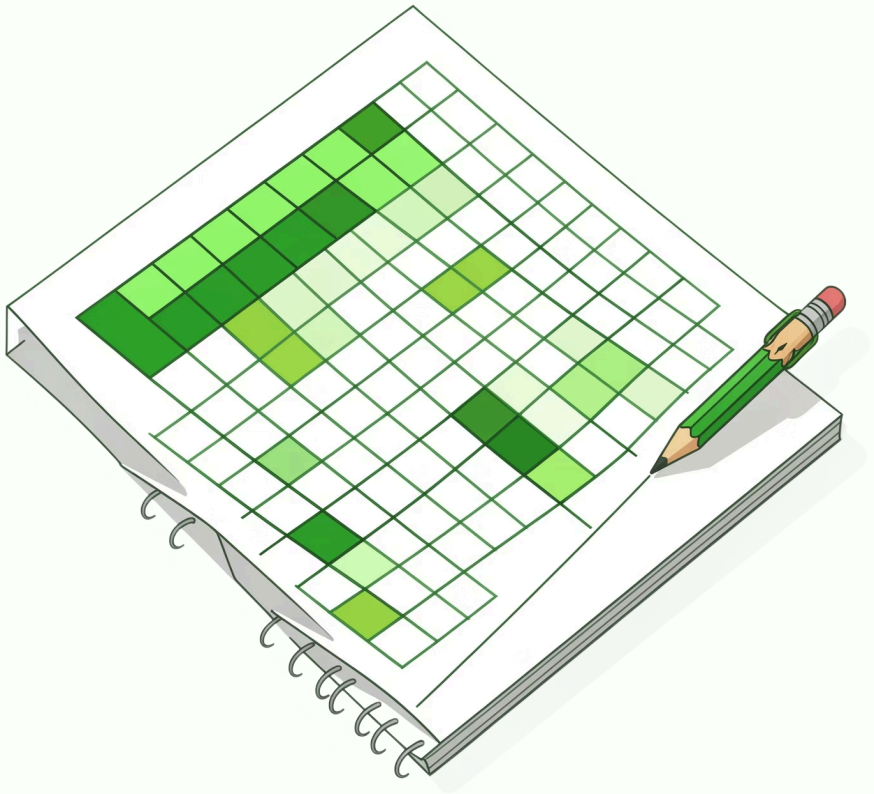
Made with GAMMA

Active Learning: Classifying Statements

Classify each statement with a partner – *"What precisely do these sentences mean?"*

1	2	3
Always True?	**Vacuously True?**	**Open Sentence?**
$x2 \geqslant 0$ for every real number x.	"If pigs can fly, then your account won't get hacked."	$X + 5 = 10$.

Made with GAMMA



Active Learning: Try It Yourself

Task: Draw a truth table and evaluate the final values for:

$$P \land (P \to Q)$$

1 Fill in the implication column

Evaluate $P \to Q$ for all combinations of P and Q

2 Perform logical AND

Combine the first column with the implication column using $\land$

This logical form is highly famous – we will use it as our **first formal proof rule** later in the semester.

Made with GAMMA

Lecture Summary



Propositions

Strictly **True or False** declarative statements



Connectives

Operations that combine ideas: ¬, ∧, ∨, ⊕, →, ↔️



Truth Tables

Systematic tools to evaluate compounds and prove **logical equivalences**



False Hypotheses

Evaluated as **True** to protect specification integrity

Made with GAMMA

Logic as a Discrete Tool

Logic is the **structural language** of discrete mathematics – converting natural language ambiguity into computational precision.

Sets

Relations

Functions

Formal Proofs

Induction

Algorithms

Graphs

Made with GAMMA

Practice & Corequisite Lab Connection

1

Reading Assignment

Chapter 3, Sections 3.1-3.2

2

Homework Practice

Problem 3.1 (False hypothesis properties); Problems 3.6 & 3.7 (Digital circuit logic formulas)

3

Corequisite Lab

Implement logical optimizations in your **COP 3530** lab assignments



Made with GAMMA

References & Further Reading

Primary Textbook

Mathematics for Computer Science, Lehman, Leighton, & Meyer, 2018
Chapter 3, Sections 3.1 - 3.2

Course Syllabus

Discrete Structures (**COT 3100C**), Miami Dade College

Prerequisite Curriculum

COP 2800 – Java Programming

Made with GAMMA